CIIC 4030 — Programming Languages

University of Puerto Rico, Mayagüez

UPRM-Edge — Subproject 1

Lexer & Parser

1. Overview

This is the first of three UPRM-Edge subprojects. Your team will build a lexer and parser for the UPRM-Edge (.uedge) language: the common core grammar shared by every team, plus the syntax extension specific to your chosen track (Memory-Aware, Power-Aware, Precision-Aware, or Portability-Aware). You are not implementing execution yet — Subproject 1 stops at recognizing whether a program is syntactically valid, and where exactly it breaks if it isn't.

Subproject 2 will build the interpreter on top of what you submit here, so the structure you choose now (your parse tree / AST shape) is something you will live with for the rest of the semester.

2. What You Must Build

2.1 Command-Line Interface

Your program must accept a single UPRM-Edge source file as a command-line argument:

```
python your_parser.py program.uedge
```

No interactive input, no hardcoded filenames. This is how the autograder will invoke your program, so this exact calling convention is required, not optional.

2.2 Lexer

Tokenize the full common core: variables/keywords (let, func, if, else, while, for, return, print), identifiers, integer and float literals, boolean and string literals, arithmetic/comparison operators (+ - * / % == != < > <= >=), punctuation, and // comments (ignored). Then extend your token set to cover your track's syntax (e.g., @memory, @power, @target annotations, or precision type keywords like int8/float16 — see your Track Selection Guide).

2.3 Parser

Implement a PLY.yacc grammar that accepts any syntactically valid combination of the common core plus your track's extension, and rejects anything that isn't — including malformed or missing track-specific syntax.

2.4 Output on Success

A clear indication the file parsed successfully. This can be as simple as printing "***Parse successful***".

2.5 Output on Failure — Required Error Format

This is graded explicitly, not a nice-to-have. On any lexical or syntax error, your program must:

- Report a specific, human-readable description of what went wrong (not just "Syntax error").
- Report the exact line number where the error occurs.
- Exit cleanly — no unhandled Python tracebacks.

Example format (yours does not need to match exactly, but must carry the same information):

```
Error: unexpected token ';' at line 14
Error: illegal character '$' at line 7
Error: malformed @memory annotation — expected '(' at line 3
```

3. Test Cases & Grading

A set of UPRM-Edge (.uedge) source files will be provided, each with an expected output, for both the common core and your track's extension. Some will be valid programs; others will contain deliberate errors your parser must catch, at the correct line.

- Grading is autograded via Moodle against these hidden test cases.
- A sample/public subset will be available before the deadline so you can validate your submission format.

4. Oral Report

Each team presents for 10 minutes. This presentation covers your lexer and parser for the common core grammar only — do not present your track-specific syntax extension here; that will be part of a later presentation once the interpreter and track enforcement are built. No live demo: this is a slide presentation.

Suggested Slide Content

1. Title — team name and members.
2. Common Core Grammar Overview — your BNF for the shared core (condensed, not every derivation step).
3. Lexer Design — your core token list and any non-obvious regex choices.
4. Parser Design — how your BNF became PLY.yacc rules; call out how you resolved any ambiguity (precedence declarations vs. grammar stratification).
5. Error Handling Design — explain your error-reporting approach and show 1–2 example messages (as text/screenshots, not a live run).
6. Challenges & Design Decisions — what was hardest, and what you'd reconsider knowing what you know now.
7. Next Steps — one-line preview of how this grammar feeds your Subproject 2 interpreter.

Aim for roughly one minute per slide — keep Slides 1–3 brief since every team shares that ground; spend the extra time on Slides 4–6, where your team's decisions actually differ from everyone else's.

5. Key Dates

Milestone	Date
Subproject 1 submission deadline (VPL)	Friday, October 16, 2026
Oral reports	Thursday, October 22, 2026 (overflow: Tuesday, October 27, 2026)

6. Submission Checklist

- Code submitted through Moodle, runnable exactly as: python your_parser.py <file>.uedge
- Lexer covers common core tokens + your track's tokens.
- Parser accepts valid common core + track syntax, rejects invalid syntax with a specific error and line number.
- Slide deck (core grammar only, no demo) ahead of your presentation slot.
- Team ready to present for 10 minutes on October 22 (or October 27, if extended).